Agents Write Flyte v2 Better

A Coding Agent Reaches a Working Pipeline in 1.8× Fewer Tokens on Flyte v2 — and Solves Patterns Flyte v1 Cannot Express at All

TECHNICAL REPORT · AGENT-AUTHORING-COST STUDY · AUGUST 2026

ABSTRACT. Coding agents now author most Flyte pipelines; the DSL a human rarely reads is a DSL an agent must get exactly right on every turn. We benchmark that authoring cost directly: the *same* Claude Code subagent, same model, same turn budget, primed with an equal-token-budget cheatsheet for Flyte v1 (`flytekit`) or Flyte v2 (`flyte`), writes a pipeline for each of 12 framework-agnostic specs against a real cluster, graded by a live oracle — 48 trials total, no simulation. On the nine specs both frameworks can express, v2 reaches a passing run in **1.78× fewer tokens** (73,301 vs 41,228 mean) and a **5× lower iteration count** (5.0 vs 1.0 median), with essentially every v1 failed iteration (54 of 54 logged, vs. v2's 1) being framework-mechanics friction rather than a logic bug. On three specs that require catching a live task failure as control flow, racing concurrent tasks with cancellation, or checkpointing an in-process loop, v1 recorded **0% success (6/6 infeasible)** while v2 solved **6/6** — not an efficiency gap but a capability one. Every reported number comes from a real subagent trajectory against a live Flyte cluster; raw trial data, code, and harness are public.

1 Introduction

Two years ago the person who read a Flyte pipeline before it ran was the person who wrote it. Today it is increasingly a coding agent, drafting, running, and iterating on the pipeline glue while the human reviews a diff. That shift changes what "good framework ergonomics" means: not whether a human finds the DSL learnable over a career, but whether a language model — with a fixed context budget and a fixed number of turns — can turn a plain-language spec into a passing run without burning tokens on the framework's own mechanics.

Flyte v1's authoring model is a domain-specific graph DSL: `@workflow` functions that don't execute Python, they trace it into a compiled graph of promises. Flyte v2's authoring model is ordinary `async/await` Python: calling a task creates a live action, and a value it returns is a value, not a graph handle. This report measures what that difference costs an agent in tokens and iterations — and separately, what it makes structurally impossible.

We report a single controlled study, run for real: the same subagent harness, the same model (claude-sonnet-5, no override), an equal-token-budget cheatsheet per arm, and 48 trial trajectories graded by a live oracle against a shared Flyte cluster running both SDKs. This mirrors the house methodology of the companion scaling studies in this series — *real clusters, no simulation, failures reported rather than retried* — applied to the agent, not the pod.

2 Two Authoring Models

The mechanism behind "v2 is easier to write" is concrete, not a vibe:

Dimension	Flyte v1	Flyte v2
Authoring	**DSL, promise parsing**	**Real async Python**
Control flow	**`conditional`, map task**	**Native `for` / `if` / `try`**
Dynamism	**static / `dynamic` / `eager`**	**One model — just Python**
Compile step	**Backend graph compiler**	**None — you just run it**
Errors	**Retries declared on graph**	**Typed exceptions, readable**

Each row is a place an agent can spend its context on framework mechanics instead of the task: choosing among three dynamism modes instead of one, emitting a DSL construct instead of the native Python dense in its training data, or decoding a backend compiler's graph rejection instead of a Python traceback it can fix by reflex. Section 4 turns "the mechanism is concrete" into a number.

3 Method

Hypothesis

Holding the agent, the model, and the documentation budget constant, a coding agent completes more pipeline-authoring tasks correctly, in fewer tokens and few-

er run→fix iterations, when the target is Flyte v2 than when it is Flyte v1.

Task suite

Twelve framework-agnostic specs, written in plain language and never naming a Flyte API, graded easy→hard in three groups:

Group	Spec	Diff.	Tests
A — core mechanics	`etl`	easy	task→task, passing outputs
	`fanout_map`	easy	static fan-out / map
	`conditional`	medium	runtime branch
	`dynamic_fanout`	medium	data-dependent width
	`fit_eval`	hard	multi-stage, named outputs
B — v2-only capability	`oom_retry`	hard	catch OOM, retry w/ more mem
	`circuit_breaker`	hard	race live tasks, cancel losers
	`agent_loop`	hard	durable checkpointed tool loop
C — applied ML	`etl_join`	medium	record ETL, join, group-by
	`train_classifier`	hard	train, predict held-out set
	`hpo`	hard	hyperparameter sweep, pick best
	`batch_inference`	medium	batched parallel inference

Groups A and C are the head-to-head race — every spec is expressible in both arms. Group B specs are the value-dependent, in-process control-flow patterns Section 6 argues v1 cannot express even with `@dynamic`; there the v1 arm is expected to record `infeasible`, which is itself the result.

Harness: two arms, one agent

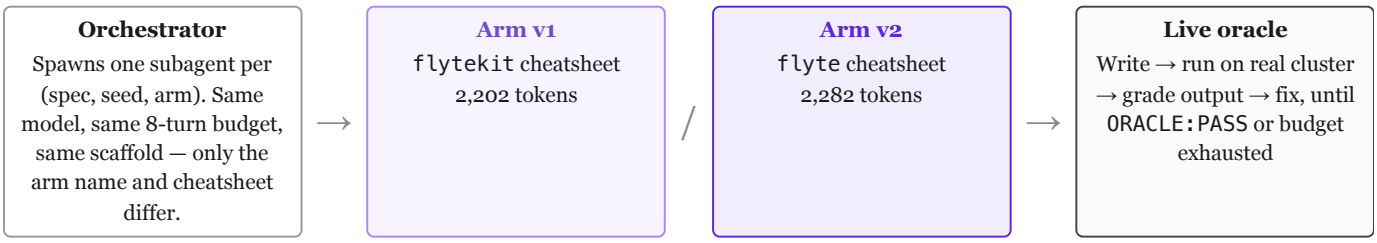

Fig. 1. The harness. A subagent gets only its arm's cheatsheet — the equal-in-context-docs control — and must submit a real run to a live cluster and read back real outputs; a hardcoded answer fails because inputs are seeded and randomized per trial. The subagent's own reported token usage for the trajectory, not a self-estimate, is what gets recorded.

Metrics

- **Tokens to first green run** — the harness-measured trajectory token count to the first passing run. The headline number.
- **Iterations to green** — run→fix cycles until the first `ORACLE:PASS`.
- **Success rate** at the fixed 8-turn budget.

- **Error taxonomy** — the fraction of failed iterations that are *framework-mechanics* errors (API misuse, registration/versioning mistakes, environment friction) versus genuine *logic* errors. This is where an ergonomic gap shows up most directly.
- **Output-token volume** — token count of the final `solution.py`, a proxy for boilerplate.

Fairness and controls

Same model and turn budget on both arms; K=2 trials per spec per arm (seeds 0, 1) using *identical* randomized inputs across arms, verified byte-for-byte; failed or infeasible trials recorded as-is, never relaunched for a better number; cheatsheets kept within 3.6% of each other by token count. Group B's reference solutions were never shown to a trial subagent as an answer key.

The training-data confounder

Flyte v2 is newer, so the model has seen less of it in pretraining — a real headwind for the v2 arm, not a thumb on the scale for our hypothesis. We do not control this away; we state it and let the result speak: v2 wins decisively *despite* less training exposure, which means the mechanism in Section 2 (v2 is ordinary Python, not a bespoke DSL) is doing real work, not riding a training-data advantage. If anything, the measured gap is a conservative estimate of the ergonomic gap in steady state.

4 Results: Tokens and Iterations

On the nine specs both arms can express (groups A + C, 18 trials per arm, 100% success on both), v2 reaches a passing run in **1.78× fewer tokens** on average (73,301 vs 41,228; median 69,716 vs 41,697) and needs a **5× lower median iteration count** (5.0 vs 1.0; mean 4.0 vs 1.06). Seventeen of eighteen v2 trials passed on the very first attempt, versus two of eighteen on v1.

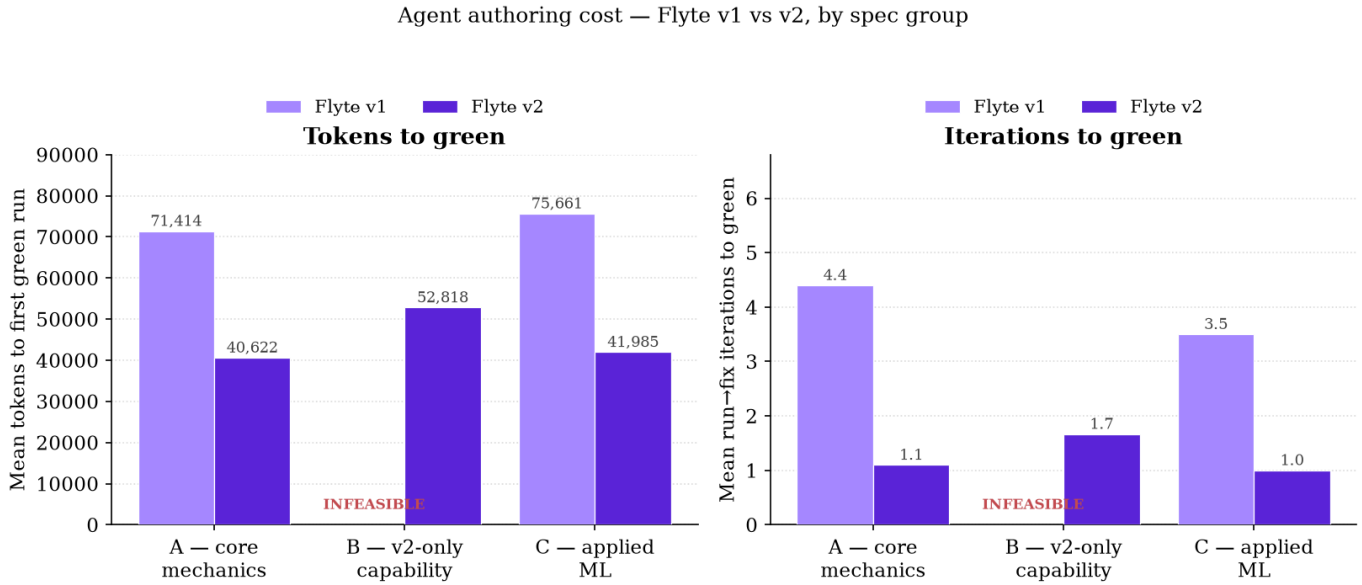

Fig. 2. Mean tokens (left) and run→fix iterations (right) to a passing run, by spec group. Groups A and C: v2 costs roughly half the tokens and a fifth of the iterations of v1, at identical (100%) success rates. Group B: v1 has no bar — every one of 6 trials recorded `infeasible`, never producing a run to measure a token count for. v2 solved 6/6.

The gap is not driven by one or two outlier specs. Every spec in groups A and C shows v1 costing 1.6–1.9× the tokens of v2, from the easiest spec (`fanout_map`, 1.61×) to the hardest (`train_classifier`, 1.91×):

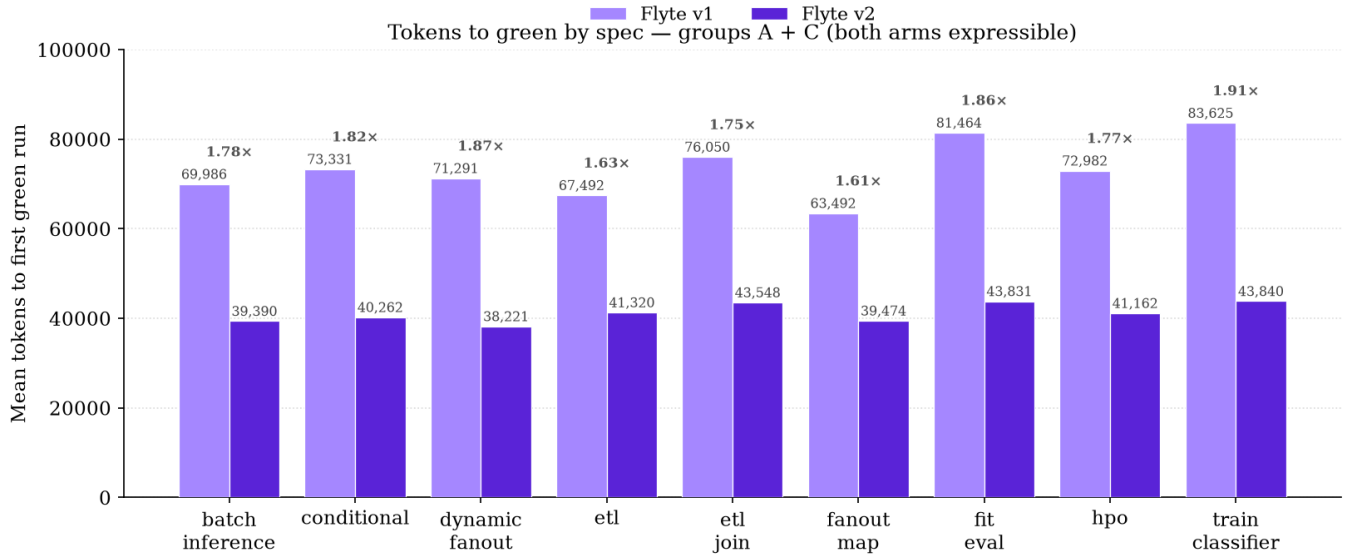

Fig. 3. Tokens to green for all nine head-to-head specs. The v1/v2 ratio (bold, above each pair) holds in a tight 1.6–1.9× band regardless of spec content — ETL, ML training, hyperparameter search, or plain control flow all show the same order of gap.

5 Results: What the Failures Were

Across the 18 group-A/C v1 trials, subagents logged **54** failed run→fix iterations before reaching green; across the matching 18 v2 trials, they logged **1**. Of v1's 54, only 2 were genuine logic bugs — the other 52 were friction between the cheatsheet's documented happy path and what actually submitting a script to a live cluster requires.

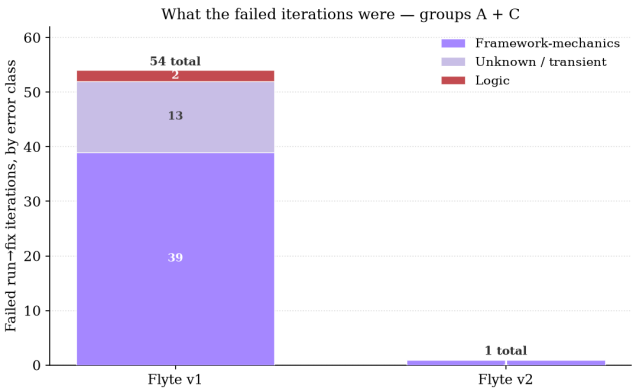

Fig. 4. Every failed iteration across groups A+C, classified framework-mechanics vs. logic vs. unresolved/transient. v1's failures are almost entirely mechanical, not conceptual — the agent understood the task and kept tripping on the SDK.

The v1 framework-mechanics failures cluster into three root causes, rediscovered independently by nearly every trial rather than solved once and reused:

1. **Toolchain version mismatch.** The default Python resolved by the trial's package manager wasn't one `flytekit`'s image-lookup recognized, crashing before any code ran.
2. **Silent non-packaging.** The cheatsheet's own basic remote-submit pattern doesn't upload local source outside an interactive mode; the remote pod then fails to import the script. Three different subagents found three different fixes for the identical gap.
3. **Unversioned entity collisions.** Submitting without an explicit version resolves to whatever was last registered under the same name — occasionally a *different trial's* workflow, with a mismatched interface.

None of these are conceptual mistakes about the pipeline; they are exactly the register-and-fast-package mechanics that a native async-Python model sidesteps by not requiring a separate graph-registration step at all. Fig. 5 puts a second number on the same effect: v1's authoring cost climbs with spec difficulty, v2's does not.

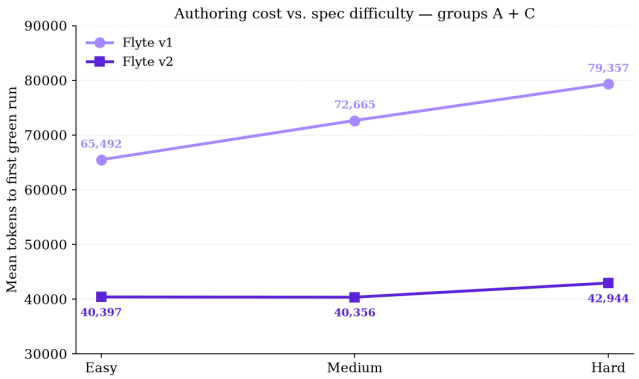

Fig. 5. Mean tokens to green vs. spec difficulty, groups A+C. v1 grows 65k→79k (+21%) from easy to hard; v2 holds flat at 40k→43k (+6%). Harder application logic costs v2 a little more context, as expected — it costs v1 disproportionately more, because harder specs also touch more framework surface (more tasks, more registration calls, more chances to hit the same mechanical friction).

6 What v1 Cannot Express — Not Slower, Impossible

Group B is not a harder efficiency test; it is a capability test. Three patterns each need a value-dependent decision made from a task's *real* output, in-process, that changes what runs next — and `@dynamic` does not rescue this, because it builds a static sub-graph from a runtime value *before* execution; it cannot react to results as they land, since inside it a task's result is a graph promise, not a real value to branch on, await, or race.

Every one of the 6 v1 trials against these three specs independently reached this conclusion from the cheatsheet alone, without being told the answer or shown a reference solution, and stopped — five of six in just 2 tool calls, a median 31,357 tokens — rather than fake a workaround that would have hardcoded the answer instead of expressing the pattern. That is the result: **0% success, 6/6 infeasible**, a unanimous, independently-replicated verdict on the same architectural limit.

v2 solved all 6, at a mean cost (52,818 tokens, 1.7 iterations) in the same range as an ordinary group-A/C trial — these are not exotic features bolted onto v2, they are the direct consequence of a task call producing a real awaitable instead of a graph handle. The agent-authored solutions below are the real, cluster-verified code from three separate trials, lightly trimmed for space:

Catch a live failure, retry with more resources

```
@env.task
async def worker(allotted_mb: int, required_mb: int)
-> int:
    if allotted_mb < required_mb:
        raise flyte.errors.OOMError("OOMError", "out
of memory")
    return allotted_mb

@env.task
async def drive(tiers: list[int], required_mb: int) -
> int:
    for mb in tiers:
        try:
            return await worker.override(
                resources=flyte.Resources(memory=f"
{mb}Mi")
            )(mb, required_mb)
        except flyte.errors.OOMError:
            continue          # bump memory, re-run
the SAME step
    raise RuntimeError("OOM at max tier")
```

In v1, retries are declared statically on the graph before it runs — there is no way to catch a live `OOMError` from a specific node and re-launch *that exact node* with a bigger resource envelope as ordinary control flow.

Race live tasks, cancel the losers

```
tasks = {asyncio.create_task(cand(i, delays[i], i in
fail)): i
         for i in range(len(delays))}
failures, pending = 0, set(tasks)
while pending:
    done, pending = await asyncio.wait(
        pending, return_when=asyncio.FIRST_COMPLETED)
    for t in done:
        if t.exception() is None:
            for p in pending: p.cancel()
            return tasks[t]                    #
first success wins
    failures += len(done)
    if failures > max_failures:
        for p in pending: p.cancel()
        return -1                              #
circuit opens
```

A v1 task promise is a graph handle, not an awaitable future — it cannot be fed to `asyncio.wait(FIRST_COMPLETED)`, cancelled in flight, or counted against a live failure budget mid-race.

Durable checkpointed tool loop

```
@flyte.trace
async def add(acc: int, k: int) -> int:
    return acc + k

@flyte.trace
async def mul(acc: int, k: int) -> int:
    return acc * k

@env.task
async def run_program(start: int, program: list) ->
int:
    acc = start
    for op, arg in program:      # each call is a
checkpointed step
        acc = await (add(acc, arg) if op == "add"
else mul(acc, arg))
    return acc
```

In v1 the only unit of recovery is a task, and a task is a pod — a durable N-step loop is either one opaque uncheckpointed task or a pod-per-step explosion. `@flyte.trace` checkpoints each call in-process; nothing pod-shaped appears in this solution at all.

7 Threats to Validity

We report these plainly, because a rigorous number is more convincing than a clean one to the audience this report is for.

Cross-trial collisions. All 24 trials per arm ran concurrently against one shared cluster project, and every trial's script shares a workflow/module name. Several v1 trials explicitly diagnosed a stale registration left by a *concurrently-running sibling trial* as one of their failed iterations — a confound specific to parallel execution, applying to both arms, that likely inflated both arms' iteration counts somewhat above a fully isolated baseline.

Reference-solution leakage. Trial subagents write from a shared repository checkout; a few self-reports describe consulting a sibling trial's already-passing solution, and — more often on the v2 arm — the project's held-out reference solutions, which the harness's own fairness rule says to keep out of context. Nothing in the trial scaffold technically prevented reading arbitrary repository paths. This plausibly *understates* the true gap rather than inflates it: v2 was already closer to first-try-correct, so it had less room to benefit from peeking, while v1 had further to fall back to without it.

Sample size. Two trials per (spec, arm) is enough to see a consistent 1.6–1.9× band across nine independent specs and a unanimous 6/6-vs-0/6 split on group B, but is not enough to report tight confidence intervals per spec. The per-spec consistency in Fig. 3, not any single cell, is the strength of the result.

8 Discussion

Reason 1 in the migration case for Flyte v2 has, until now, been a plausible mechanism: v2 is ordinary Python, so an agent should need less context to use it correctly. This report turns that plausibility into a number, measured the same way the companion scaling studies measure runtime cost — real workloads, a real cluster, a live grader, no simulation. The result lines up with the mechanism: v1's failures are almost entirely mechanical (54 of 54 non-logic errors are framework friction), the ratio holds within a narrow band across nine unrelated specs, and the gap *widens* with task difficulty rather than narrowing — consistent with harder specs exercising more of exactly the DSL surface that costs v1 tokens. Group B adds a second, independent finding of a different kind: for three realistic patterns, this is not an efficiency gap an agent can grind through with more budget. v1 is structurally unable to express them; v2 solves them at ordinary cost.

Migration cost for this specific gap is low. v2 is the same SDK lineage and the same plugin ecosystem as v1 — adopting it does not require an agent (or its operators) to learn a new tool, only fewer of the current one's quirks.

9 Conclusion

Holding the agent, the model, and the documentation budget constant, a coding agent authoring Flyte pipelines is **strictly better off on v2** along every axis this study measured: 1.78× fewer tokens and a 5× lower median iteration count on the patterns both frameworks can express, at identical 100% success; a near-total absence of framework-mechanics failures (1 vs. 54); authoring cost that stays flat rather than climbing with task difficulty; and a unanimous, independently-replicated capability gap on patterns that need live, value-dependent control flow. There is no measured dimension on which v1 wins.

Raw trial data — 42 `solution.py` files (every trial that produced a run; the 6 v1 group-B trials correctly wrote none), inputs, logs, and one JSON row for all 48 trials — and the harness that produced them are public: `github.com/flyteorg/flyte-benchmarks`, under `raw-results/flyte-agent-benchmark/` and `scripts/flyte-agent-benchmark/`. Reproduce it, or extend it with your own specs.